NORTHERN TECHNICAL UNIVERSITY

Technical Engineering College for Computer and AI / Mosul
Department of Cybersecurity and Cloud Computing Techniques Engineering

# SECURESENSE-24

## An Offline Dual-Sensor Human-Presence and Wireless Channel-State Analysis Platform

**INTEGRATED COURSE PROJECT | THREE DISCIPLINES**

| Target course | Focus in this project |
|---|---|
| Digital Signal Processing (DSP) | Signal filtering, robust detection, temporal FFT, MATLAB verification |
| Wireless Networks | 2.4 GHz Wi-Fi CSI acquisition, 24 GHz radar and local networking |
| Cryptography | AES-256-GCM authenticated-encryption laboratory and integrity tests |

**Prepared by**
Mohammed Younis Majid

Academic Year 2026-2027 | Mosul, Iraq | October 2026

*Technical project documentation; evidence and test limitations are identified explicitly.*

# Project at a Glance | Components & CSI

This short orientation identifies the actual hardware and software stack before the research methods, signal-processing equations, and code-level implementation. SECURESENSE-24 operates locally and does not require Internet access.

## A. Hardware components

| Component | Function in SECURESENSE-24 |
|---|---|
| ESP32-S3 development board | One controller reads the radar over UART, receives Wi-Fi CSI and sends SS24 JSON telemetry to Windows over USB. |
| HLK-LD2410B (24 GHz radar) | Reports processed presence / motion states, approximate distance, and moving / stationary energy values. |
| TP-Link TL-WR940N v6 router | Provides the local 2.4 GHz Wi-Fi network and traffic used for CSI reception; no WAN/Internet needed. |
| Windows computer + USB cable | Powers/communicates with the microcontroller and runs the offline dashboard. |

## B. Software components

| Software / tool | Purpose |
|---|---|
| ESP-IDF 6.1 + C | Firmware: radar frame parsing, Wi-Fi CSI reception, gateway Ping, and serial JSON output. |
| Python + pyserial | Local serial-to-HTTP bridge, CSI motion classifier, experiment logs and CSV export. |
| HTML / CSS / JavaScript | Browser interface, live waveform displays, filtering, and short-window Fourier analysis. |
| cryptography (Python library) | PBKDF2 key derivation and AES-256-GCM encryption/decryption demonstration. |
| MATLAB (analysis companion) | Optional reproducible analysis of CSV exports; not required to operate the live dashboard. |

## C. What is Wi-Fi CSI, and how does it work?

**Channel State Information (CSI)** describes how a Wi-Fi propagation channel modifies a known radio signal across OFDM subcarriers. Reflections, obstacles and motion can alter the estimated complex channel response, so its amplitude and phase may change with the environment. Unlike RSSI (one aggregate received-strength measure), CSI exposes a more detailed set of per-subcarrier channel estimates.

**In this project:** the TP-Link router transmits local Wi-Fi frames; the ESP32-S3 receives them and the Wi-Fi driver exposes CSI I/Q measurements. Firmware derives amplitude samples and streams compact vectors over USB to Python. The dashboard measures changes between consecutive vectors to produce a relative motion index.

**Engineering distinction:** CSI can respond to walking, but changes are not unique to people. The LD2410B independently provides a presence/radar observation. The CSI-derived temporal FFT is not a measurement of the 2.4 GHz RF spectrum.

# Executive Abstract

SECURESENSE-24 is an offline laboratory prototype that combines an ESP32-S3 microcontroller, a 24 GHz HLK-LD2410B presence radar, and channel-state information (CSI) derived from received 2.4 GHz Wi-Fi frames. A Windows-based localhost application interprets newline-delimited serial telemetry, displays temporal signal behavior, and provides an AES-256-GCM cryptography laboratory. The system was designed as one integrated practical project for Digital Signal Processing, Wireless Networks, and Cryptography.

The radar provides processed presence/motion states and approximate range. Wi-Fi CSI supplies a separate measurement of changing propagation conditions. The v1.4 Windows software implements a relative, robust CSI change feature, a 12-second quiet calibration, an adaptive threshold with hysteresis, and an operator-labeled evaluation protocol. Sensor fusion combines the two signals conservatively and does not equate CSI variation with definite human presence.

**Evidence status:** the student has observed live radar and CSI telemetry reaching Windows and changing under movement in an earlier working version. The final v1.4 classifier has passed software tests, but operator-labeled field results for the three evaluation phases have not been supplied. Accordingly, this report presents no invented detection-accuracy percentage.

**KEYWORDS | ESP32-S3 | HLK-LD2410B | Wi-Fi CSI | DSP | AES-GCM**

# Contents

Project at a Glance | Components & CSI 2

Executive Abstract 3

Contents 4

01 | Research Scope & System Architecture 5

02 | Acquisition, Software & Evidence 6

03 | Digital Signal Processing: Radar 7

04 | Digital Signal Processing: FFT & CSI 8

05 | DSP: Adaptive CSI Motion Classifier 9

06 | DSP: Code Explanation & MATLAB Exercise 10

07 | Wireless Networks: Router & CSI 11

08 | Wireless Networks: Measurement Integrity 12

09 | Cryptography: Secure Payload Laboratory 13

10 | Code Integration & Sensor Fusion 14

11 | Experimental Design & Evaluation Metrics 15

12 | Application Interface: Radar Monitor 16

13 | Application Interface: Dual-Sensor Fusion 17

14 | Application Interface: DSP Laboratory 18

15 | Application Interface: CSI DSP 19

16 | Application Interface: Wireless/CSI 20

17 | Application Interface: Validation 21

18 | Application Interface: Cryptography 22

19 | Student-Observed Field Evidence 23

20 | MATLAB Appendix: Measured CSV Analysis 24

21 | Limitations, Conclusion & Recommendations 25

References & Source Provenance 26

Notation: [O] = observation supplied by student, [S] = software implementation or automated test, [D] = demonstration preview, [P] = planned field validation.

# 01 | Research Scope & System Architecture

## 1.1 Problem statement and objectives

Indoor motion sensing based on a single modality is subject to environment-dependent ambiguity. SECURESENSE-24 therefore records two physically distinct observations: a dedicated 24 GHz presence radar and estimated Wi-Fi channel changes on a local 2.4 GHz network. The objective is a demonstrable, auditable educational laboratory rather than a certified security alarm.

- Acquire radar and router CSI concurrently from one ESP32-S3 while preserving a fully offline architecture.
- Display processed radar motion energy, stationary energy, distance, CSI amplitude, and temporal FFT with correct physical labels.
- Develop a calibrated, noise-adaptive CSI motion indicator and compare it with radar outputs.
- Demonstrate authenticated encryption and integrity checking on local telemetry snapshots.
- Export CSV evidence and distinguish software tests from controlled field validation.

## 1.2 End-to-end engineering architecture

| | | |
|---|---|---|
| SENSING LAYER | | |
| 24 GHz HLK-LD2410B | UART1, 256000 baud | ESP32-S3 |
| 2.4 GHz TP-Link TL-WR940N | Wi-Fi AP + local ICMP echo | ESP32-S3 |
| DATA TRANSPORT | | |
| ESP32-S3 firmware | USB serial, 115200 baud | SS24: {JSON} lines |
| WINDOWS ANALYSIS | | |
| Python backend | Classification / evaluation / crypto | Local HTTP 127.0.0.1:8765 |
| Browser dashboard | Monitor, DSP, Wireless, Crypto | CSV export |

The router does not need an Internet/WAN connection. The ESP32-S3 supplies sensor telemetry to Windows through USB; CSI is not collected from a conventional Windows Wi-Fi adapter. The Windows adapter is used separately for a local BSSID survey.

## 1.3 Main components

| Subsystem | Implemented role | Qualification |
|---|---|---|
| ESP32-S3 | Unified radar UART and Wi-Fi CSI receiver | One MCU; not three |
| LD2410B | Presence, movement/stationary energy, range | No angle or multi-person count |
| TP-Link TL-WR940N v6 | 2.4 GHz access point / local stimulus | No Internet necessary |
| Windows + Python | Serial bridge, calibration, UI API, crypto | Localhost only |

# 02 | Acquisition, Software & Evidence

## 2.1 Hardware/firmware data path

The radar transmits 23-byte engineering frames on UART1 at 256000 baud. The firmware validates frame header/trailer bytes and parses target state, moving and stationary distances, and energy values. Simultaneously, Wi-Fi station mode associates with the local router. The program receives CSI callbacks from the Wi-Fi driver, filters frames by the access-point BSSID, computes magnitude slots from signed I/Q byte pairs, and transfers compact samples through a queue to a lower-priority output task.

A local ICMP echo to the router gateway at approximately 120 ms intervals provides recurring received frames. Firmware limits published CSI to about eight frames/s to control serial traffic; the observed Windows CSI rate can be lower. The serial messages use a stable `SS24:` prefix so the Windows backend can ignore unrelated console logs.

## 2.2 Frame schema and provenance

```
SS24:{"event":"radar","seq":261,"moving":true,...}
SS24:{"event":"wifi","connected":true,"channel":11,...}
SS24:{"event":"csi","seq":105,"rssi":-48,"channel":11,
      "subcarrier_amplitude":[0,10,12,...]}
```

The examples above illustrate the telemetry **format**; they are not a freshly recorded raw log or a guarantee of particular amplitudes. Radar and CSI sequence counters are independent. Missing USB/serial application sequence numbers must not be interpreted automatically as lost IEEE 802.11 frames.

## 2.3 Software components

| File or module | Responsibilities |
|---|---|
| firmware/main/hello_world_main.c | UART radar parser; Wi-Fi CSI callback; gateway Ping; JSON telemetry |
| windows_app/main.py | Serial reader, HTTP API, exports, radar/CSI state and fusion |
| windows_app/csi_motion.py | Robust CSI feature, calibration, hysteresis, evaluation |
| windows_app/static/app.js | Browser plots, time-domain filter, FFT, controls and live rendering |
| windows_app/lab_tools.py | AES-GCM cryptography, PBKDF2 and Windows WLAN BSSID scan |

## 2.4 Evidence classification

| Item | Available evidence | Interpretation |
|---|---|---|
| Radar + CSI simultaneous USB stream | Student ESP-IDF monitor photograph [O] | Concurrent acquisition achieved in the tested setup |
| Live Wireless / CSI page | Student Windows photograph [O] | App received and visualized real CSI frames |
| Movement vs quiet CSI shape | Student motion/quiet trial [O] | Qualitative sensitivity; not quantified accuracy |
| v1.4 classifier + evaluation | Automated tests and demonstration preview [S/D] | Logic works in tests; independent field accuracy pending |

# 03 | Digital Signal Processing: Radar

## 3.1 What is actually measured?

The LD2410B is not used as a raw complex 24 GHz radar receiver. Its onboard processing outputs classified moving/stationary target indicators, energy values and approximate range. The Windows waveform is therefore the **time variation of a processed energy indicator**, not the transmitted 24 GHz electromagnetic carrier. A change in energy is useful for trend analysis but does not establish a calibrated microwave power level.

## 3.2 Time-domain sequence and moving-average filter

Let x[n] denote the reported radar moving-energy value at sample index n (nominal 0-100 indicator). The displayed five-sample causal moving average is:

$$y[n] = \frac{1}{M} \sum_{k=0}^{M-1} x[n-k], \qquad M = 5$$

At the beginning of a capture, fewer than five preceding samples exist, so the implementation averages the available samples. This smooths brief spikes at the cost of temporal lag; it does not recover a raw radar baseband signal.

```
% MATLAB demonstration: exported radar-energy samples
T = readtable('securesense24_radar.csv');
x = double(T.motion_energy_0_100);
y = movmean(x, [4 0]);          % causal 5-sample average
figure; plot(x, 'Color',[0.5 0.5 0.5]); hold on
plot(y, 'LineWidth',1.5); legend('Raw','5-point average')
xlabel('Frame index'); ylabel('Radar energy indicator (0-100)');
```

**MATLAB mapping:** `movmean(x,[4 0])` uses the current sample and four prior samples, matching the dashboard's causal moving-average concept. MATLAB is presented as an independent verification/analysis companion; the live browser charts are implemented in JavaScript.

## 3.3 Summary statistics

$$\mu = \frac{1}{N} \sum_{n=0}^{N-1} x[n], \qquad \sigma = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} (x[n] - \mu)^2}$$

Mean indicates average processed motion energy; population standard deviation describes within-window variability. Peak energy is the largest sample in the displayed window. None is a human-identification score.

# 04 | Digital Signal Processing: FFT & CSI

## 4.1 Sampling and frequency interpretation

Samples arrive through serial communication with time variation. Before its finite-window Fourier calculation, the browser interpolates onto an approximately uniform time grid. For radar the displayed FFT uses up to 64 points; a Hann window follows mean removal. For a uniformly spaced time sequence of effective sampling rate $f_s$, the frequency-bin interval is approximately $f_s/N$ and the Nyquist frequency is $f_s/2$.

$$X[k] = \sum_{n=0}^{N-1} w[n](x[n] - \bar{x})e^{-j2\pi kn/N}$$

$$w[n] = \tfrac{1}{2}\left(1 - \cos\left(\tfrac{2\pi n}{N-1}\right)\right), \qquad f_k = \tfrac{kf_s}{N}$$

The plotted magnitude is approximately |X[k]|/N. The current implementation evaluates the discrete Fourier sum directly (a DFT) for short windows, rather than invoking a radix-2 FFT library; the mathematical frequency content is equivalent. These are **temporal modulation frequencies**, not a scan over the 2.4 GHz or 24 GHz radio-frequency band.

## 4.2 MATLAB reproduction of the temporal spectrum

```
% Example using radar CSV; interpolate irregular samples
T = readtable('securesense24_radar.csv');
t = double(T.time_monotonic_s);
x = double(T.motion_energy_0_100);
[t, i] = unique(t, 'stable'); x = x(i);
N = 64; tq = linspace(t(1), t(end), N)';
xq = interp1(t, x, tq, 'linear');
fs = (N-1)/(tq(end)-tq(1));
X = fft((xq-mean(xq)).*hann(N));
f = (0:floor(N/2)-1)'*fs/N;
plot(f, abs(X(1:numel(f)))/N); xlabel('Temporal Hz');
ylabel('Windowed energy magnitude'); grid on
```

Requires a nonempty recording with increasing timestamps and sufficient duration. If samples are sparse or irregular, interpolation can distort fast events; interpretation should respect the observed sample rate and avoid claims beyond its Nyquist limit.

## 4.3 CSI amplitudes versus instantaneous channel response

The Wi-Fi driver provides signed in-phase and quadrature estimates for received training fields. Firmware computes amplitude values from each selected I/Q pair; Windows then analyzes both instantaneous amplitude distribution and changes over time. The firmware outputs up to 52 *selected amplitude slots* drawn from available CSI pairs. Because its selection may subsample or mix supported training-field ranges, exported slot indices should **not** be treated as calibrated, one-to-one absolute OFDM subcarrier frequencies without further mapping.

$$a_i = \sqrt{I_i^2 + Q_i^2}, \qquad \bar{a}_t = \frac{1}{K}\sum_{i=1}^{K} a_{t,i}$$

# 05 | DSP: Adaptive CSI Motion Classifier

## 5.1 Robust inter-frame variation feature

For consecutive amplitude vectors of identical length and channel, define a baseline-normalized change feature $D_t$. The median of absolute per-slot differences reduces sensitivity to isolated corrupted slots; a smaller contribution from the overall mean change retains information about broad amplitude shifts.

$$D_t = \frac{\mathrm{med}_i |a_{t,i} - a_{t-1,i}| + 0.25\,|\bar{a}_t - \bar{a}_{t-1}|}{\max(1, \mathrm{med}_i\, a_{t-1,i})}$$

The feature is dimensionless. Samples are rejected if malformed, nonfinite, negative, shorter than eight slots, or incompatible in channel/length; comparisons also restart after a gap over two seconds. The code caps $D_t$ at 10 to limit extreme outliers.

## 5.2 Exponential smoothing and quiet-room calibration

$$S_t = \alpha D_t + (1 - \alpha) S_{t-1}, \qquad \alpha = 0.55$$

A nominal 12-second quiet calibration gathers at least 24 inter-frame features. Let m be their median and $\mathrm{MAD}_r$ = 1.4826 × median($|D_t - m|$). The implemented baseline/noise proxy and trigger threshold are:

$$B = \max(0.002, m + 2\,\mathrm{MAD}_r)$$

$$T_{on} = \max(0.015, m + 6\,\mathrm{MAD}_r, 2.2B)$$

A moving decision is asserted after two smoothed features above $T_{on}$. It clears after four values below $0.6T_{on}$. During quiet periods, the threshold slowly adapts toward a recalculated robust candidate, using 98% previous threshold and 2% new estimate. This combination of hysteresis, debouncing, and quiet-only adaptation reduces rapid false transitions.

## 5.3 Motion index (not a probability)

$$\mathrm{Index}_t = \mathrm{clip}_{[0,\,100]}\left(40\frac{S_t}{T_{on}}\right)$$

The index is an interface normalization chosen in software; a value such as 80 is **not** an 80% probability that a person moved. It is also not a physically calibrated speed or distance.

# 06 | DSP: Code Explanation & MATLAB Exercise

## 6.1 Representative v1.4 Python logic

```
# windows_app/csi_motion.py (simplified; actual constants)
den = max(1.0, median(previous_amplitudes))
delta = median(abs(a-b) for a,b in zip(curr, prev))/den
delta += 0.25*abs(mean(curr)-mean(prev))/den
ema = delta if ema is None else 0.55*delta + 0.45*ema
if ema > threshold: high_hits += 1
if not moving and high_hits >= 2: moving = True
if moving and low_hits >= 4: moving = False
```

The complete implementation includes input checks, recalibration, channel resets, low-hit reset logic, online freshness, and experiment logging. The excerpt above is explanatory pseudocode rather than a drop-in replacement.

## 6.2 MATLAB exercise on exported CSI amplitudes

```
% CSI CSV fields: timestamp, seq, rssi_dBm, wifi_channel,
% motion_feature, motion_score, motion_detected, subcarrier_0...
T = readtable('securesense24_csi.csv');
C = startsWith(string(T.Properties.VariableNames),'subcarrier_');
A = double(T{:,C});
valid = all(isfinite(A),2); A = A(valid,:);
assert(size(A,1)>30, 'Collect more CSI frames');
D = zeros(size(A,1)-1,1);
for n=2:size(A,1)
    prev=A(n-1,:); cur=A(n,:); den=max(1,median(prev));
    D(n-1) = median(abs(cur-prev))/den ...
       + .25*abs(mean(cur)-mean(prev))/den;
end
S = filter(.55,[1 -.45],D); % zero-state EMA approximation
plot(D); hold on; plot(S,'LineWidth',1.5);
legend('Normalized change','EMA'); xlabel('CSI transition');
```

The MATLAB filtering example has zero initial state and does **not** include real-time channel resets, 12-second calibration, adaptive threshold, or timestamp-based windowing. It is an intentionally simplified reproduction of the feature and smoothing math. To reproduce the live classifier exactly, use the original Python algorithm with timestamps and matching channel.

## 6.3 Why stationary plots still show small bumps

Small variations can arise from multipath, electronic noise, network timing, nearby movement and phase/amplitude estimation differences. The practical observation that plots became almost flat during a quiet interval is consistent with a stable channel, but alone cannot distinguish a stationary human from an empty room. Radar presence information improves interpretation.

# 07 | Wireless Networks: Router & CSI

## 7.1 Local network topology

| Link or component | Protocol / role | Practical observation |
|---|---|---|
| Router to ESP32-S3 | 2.4 GHz IEEE 802.11 link | CSI callback; RSSI and channel |
| Gateway to ESP32-S3 | Local ICMP echo reply | Traffic stimulus; no WAN |
| Radar to ESP32-S3 | UART1 at 256000 baud | Processed 23-byte radar frames |
| ESP32-S3 to Windows | USB UART serial at 115200 baud | Newline-delimited SS24 JSON |
| Windows to browser | HTTP on 127.0.0.1:8765 | Local dashboard and exports |

## 7.2 How CSI relates to wireless propagation

In a simplified multipath model, a wireless channel can be represented by complex path contributions. A received OFDM training symbol gives a channel estimate H(k,t) for a subcarrier index k at time t. An object moving between transmitter and receiver may change attenuation and phase via reflection, diffraction and blocking. CSI is sensitive to those changes, but so are movement of the router, antenna orientation, interference and changes in transmitter/receiver state.

$$H(k,t) = \sum_{p=1}^{P} \beta_p(t) e^{-j2\pi f_k \tau_p(t)}$$

Here $\text{beta}_p(t)$ denotes an effective complex path coefficient, $\text{tau}_p(t)$ is path delay, and $f_k$ denotes an OFDM subcarrier frequency offset. This is a theoretical model for interpretation; the application does not fit individual path delays or estimate a multipath channel model.

## 7.3 Driver acquisition and constraints

```
// firmware: Wi-Fi CSI acquisition sequence (ESP-IDF)
esp_wifi_set_csi_rx_cb(wifi_csi_cb, NULL);
esp_wifi_set_csi_config(&csi_config);
esp_wifi_set_csi(true);
// callback: reject non-router MAC; queue short IQ-derived data
// output task: emit SS24 JSON on the USB serial console
```

Espressif documents that the callback runs in the Wi-Fi task; time-consuming work should be moved to a queue. The program follows that pattern. Firmware may suppress callbacks to limit output rate, so the dashboard's CSI sample rate does not equal the underlying PHY frame rate. [1]

# 08 | Wireless Networks: Measurement Integrity

## 8.1 Channel, RSSI, and CSI are different quantities

| Quantity | Meaning | Not to conclude |
|---|---|---|
| Wi-Fi channel (e.g., 11) | AP RF channel designation | Not a CSI spectral peak |
| RSSI (dBm) | Approximate received strength metadata | Not distance or user count |
| CSI magnitude slots | Magnitude from selected signed I/Q pairs | Not calibrated RF spectrum |
| CSI frames/sec | Rate reaching Python/dashboard | Not total Wi-Fi packets per second |
| Windows BSSID survey | Nearby WLAN BSSID/channel/percent | Not a spectrum-analyzer measurement |

## 8.2 Sampling, bandwidth and observable motion

The observed v1.3 dashboard earlier showed CSI sample rates of only a few samples per second. At an effective sample rate $f_s$ = 4.6 samples/s, the nominal Nyquist limit for a uniformly sampled feature is 2.3 Hz. This number is an **illustration based on a photographed readout**, not a constant guaranteed throughput or proof that every motion process is alias-free. Because sample times are irregular, frequency estimates require additional care.

$$f_{Nyquist} = \frac{f_s}{2}, \qquad \Delta f \approx \frac{f_s}{N}$$

A larger frame rate would be useful for finer temporal analysis, but increasing the ICMP stimulus alone cannot guarantee a proportional number of valid CSI samples; firmware throttling, Wi-Fi traffic and USB logging may still dominate.

## 8.3 Information is filtered and reduced before export

The current firmware keeps up to 52 magnitude slots from CSI I/Q pairs and does not preserve every raw I/Q pair, training-field type, hardware timestamp, pilot position or complex phase. This design reduces payload size for a one-board demonstration, but limits rigorous estimation of calibrated channel response or Doppler. A research-grade extension should retain full raw CSI metadata and channel-specific calibration.

## 8.4 Student-observed qualitative result

During the student-run test, the live CSI graph became nearly flat with small residual fluctuations when the participant was still; walking increased graph variation. This supports a preliminary movement-sensitivity claim under the tested arrangement. No room-to-room specificity, population-level accuracy or verified breathing detection is claimed.

# 09 | Cryptography: Secure Payload Laboratory

## 9.1 Authenticated-encryption objective

The Cryptography page demonstrates confidentiality and integrity for a selected text payload, including an optional live telemetry snapshot. It uses AES with a 256-bit derived key in Galois/Counter Mode (AES-256-GCM). GCM performs authenticated encryption, where the authentication tag causes decryption to fail after ciphertext or associated-data modification. [2]

| Parameter | Configured value | Purpose |
|---|---|---|
| AES key | 256 bits (32 bytes) | Symmetric encryption key |
| KDF | PBKDF2-HMAC-SHA256 | Derives key from a user passphrase |
| KDF salt | 16 random bytes per encryption | Avoids deterministic key reuse across envelopes |
| PBKDF2 iteration count | 250,000 | Raises cost of password guessing |
| GCM nonce | 12 random bytes per encryption | Unique IV required for safety |
| GCM tag | 128-bit tag (library default) | Integrity/authenticity check |
| Additional authenticated data | Fixed SECURESENSE24 domain string | Binds ciphertext to lab context |

## 9.2 Data flow

**Encryption:** password + random salt -> PBKDF2 -> 256-bit key; plaintext + random nonce + AAD -> AES-GCM -> ciphertext and authentication tag; parameters are serialized as a Base64 URL-safe envelope.

**Decryption:** parse envelope -> derive same key -> AES-GCM verification -> recover plaintext only if the passphrase and authenticated contents are valid. Incorrect passphrases and tampered ciphertext result in authentication failure.

```
# windows_app/lab_tools.py (representative production code)
salt  = secrets.token_bytes(16)
nonce = secrets.token_bytes(12)
key = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32,
                 salt=salt, iterations=250_000).derive(password)
ciphertext_tag = AESGCM(key).encrypt(nonce, plaintext, AAD)
# Successful decrypt verifies the GCM tag before returning plaintext.
```

## 9.3 Scope of actual protection

**Important:** the AES-GCM lab does not automatically encrypt radar or CSI bytes travelling across the ESP32-S3 USB link, nor does it make the local HTTP endpoint independently authenticated. The Wi-Fi access-point link uses its own configured wireless encryption; that is not equivalent to the AES-GCM educational payload test. This distinction is necessary for a valid threat model.

# 10 | Code Integration & Sensor Fusion

## 10.1 Backend event dispatch

The Python process reads serial lines, parses valid JSON after the SS24 prefix, and dispatches by `event`. Radar messages update target-state and recent energy histories; CSI messages update RSSI, channel, amplitudes, rolling histories and the independent CSI classifier. The browser polls `/api/status`, with exports and cryptography requests processed by local endpoints.

```
if msg["event"] == "radar":
    # Update presence, energy, distance and radar freshness
elif msg["event"] == "csi":
    # Validate amplitude vector and channel; update classifier
elif msg["event"] == "wifi":
    # Update AP link status and IP address
```

## 10.2 Conservative rule-based sensor fusion

| Radar status | CSI motion | Displayed interpretation |
|---|---|---|
| Presence + radar moving | Either | MOTION DETECTED |
| Presence + radar still | Detected | MOTION DETECTED (cross-sensor difference may occur) |
| Presence + radar still | Not detected | STATIONARY PRESENCE |
| No radar target | Detected | CSI ACTIVITY / NO RADAR TARGET |
| No radar target | Not detected | NO TARGET |
| Radar offline | Detected | CSI ACTIVITY / RADAR OFFLINE |

The radar remains the source of evidence for an actual human-presence decision. The semicircular radar indicator is **symbolic**; the LD2410B delivers distance and state but does not report an azimuth angle in this application. The system is not capable of reliably counting multiple people.

## 10.3 Code-level verification

The Windows project contains automated tests for radar parsing, CSI ingest, AES-GCM, quiet calibration, motion triggering, channel changes, and evaluation metrics. A browser smoke test verified rendering of the principal pages in a controlled offline preview. These tests establish functional code paths only; the classifier's real-world detection performance requires independently labeled observations.

# 11 | Experimental Design & Evaluation Metrics

## 11.1 Controlled protocol

After the quiet calibration completes (approximately 12 s, at least 24 valid transitions), the operator records three 20-second phases using built-in buttons. Keep router channel, furniture, antenna placements, and participant path fixed within a run; repeat the run after meaningful geometry changes. Labeling is manual and must reflect what the participant actually does.

| Phase | Operator procedure | Ground truth for binary motion |
|---|---|---|
| EMPTY | No human in the monitored test path for 20 s | 0 = no motion |
| STILL | Participant sits/stands still for 20 s | 0 = no motion |
| MOVING | Participant walks through the Wi-Fi path for 20 s | 1 = motion |

## 11.2 Confusion matrix and derived measures

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Specificity} = \frac{TN}{TN + FP}, \qquad F_1 = \frac{2TP}{2TP + FP + FN}$$

TP, FP, TN and FN are defined for CSI **motion** decisions, not human identity. Undefined metrics with a zero denominator must be reported as N/A rather than 0%. A high sample-level accuracy can be misleading because adjacent frames are correlated. Report independent session count and environment details with aggregate metrics.

## 11.3 Evidence table for actual measurement

| Measure | Current report entry |
|---|---|
| Observed concurrent radar + CSI output | Confirmed qualitatively by user photographs |
| Movement/quiet signal response | Observed qualitatively by the student |
| v1.4 labeled session counts | PENDING - run EMPTY / STILL / MOVING |
| v1.4 TP / FP / TN / FN | PENDING - export evaluation CSV |
| v1.4 Accuracy / Precision / Recall / F1 | NOT REPORTED until measured |

# 12 | Application Interface: Radar Monitor

The Presence Monitor shows a **symbolic** semicircular range graphic, color-coded moving/stationary state, estimated distance, processed radar energies, and a local alert control. The second view exposes the independent CSI classifier and fused interpretation. Images in this section are UI captures produced under a labeled simulated-preview state; they are not photographs of the student's field measurements.

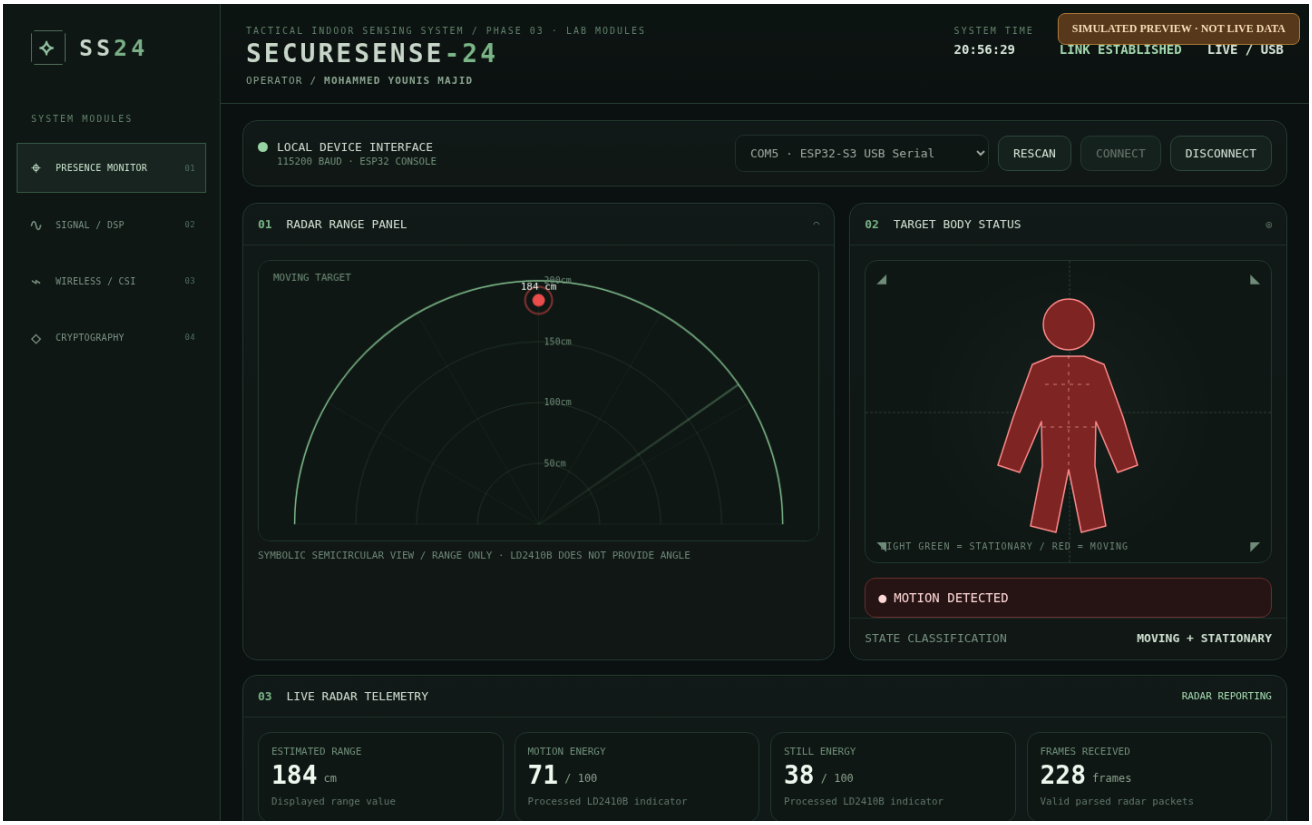



*Figure 1. Presence Monitor preview: symbolic radar range, target status, live-telemetry panel (illustrative data).*

# 13 | Application Interface: Dual-Sensor Fusion




*Figure 2. v1.4 fusion, motion index, alerts and event log (simulated UI preview).*

## Reading the monitor correctly

A red marker means the radar reports moving energy; a green marker indicates stationary presence as reported by the sensor. When CSI and radar disagree, the dashboard preserves both readings rather than overwriting one to force agreement. The Windows app continues operating locally after disconnecting external Internet.

# 14 | Application Interface: DSP Laboratory




*Figure 3. Radar energy waveform, five-point moving average, signal statistics and temporal DFT magnitude (simulated UI preview).*

The charts above provide visual demonstrations of time-domain smoothing and short-window spectral decomposition. The vertical values are processed energy indicators, and horizontal spectral frequency is temporal Hz.

# 15 | Application Interface: CSI DSP




*Figure 4. CSI motion feature versus adaptive threshold and temporal CSI amplitude/DFT views (simulated UI preview).*

The classifier plot compares normalized relative frame-to-frame CSI variation against a calibrated threshold. Neither this plot nor the mean-amplitude DFT can be interpreted as a direct 2.4 GHz RF spectrum.

# 16 | Application Interface: Wireless/CSI




*Figure 5. Wireless/CSI page: WLAN survey, router status, frame counters, received signal strength, and CSI amplitude curve (simulated UI preview).*

## What each indicator tells us

The Windows wireless survey relies on OS-reported BSSIDs and channel values; the ESP32-S3 produces the CSI curve separately. The RSSI entry is a relative reception-strength indicator. Frame sequence and count entries describe captured application messages rather than a calibrated network quality-of-service test.

# 17 | Application Interface: Validation

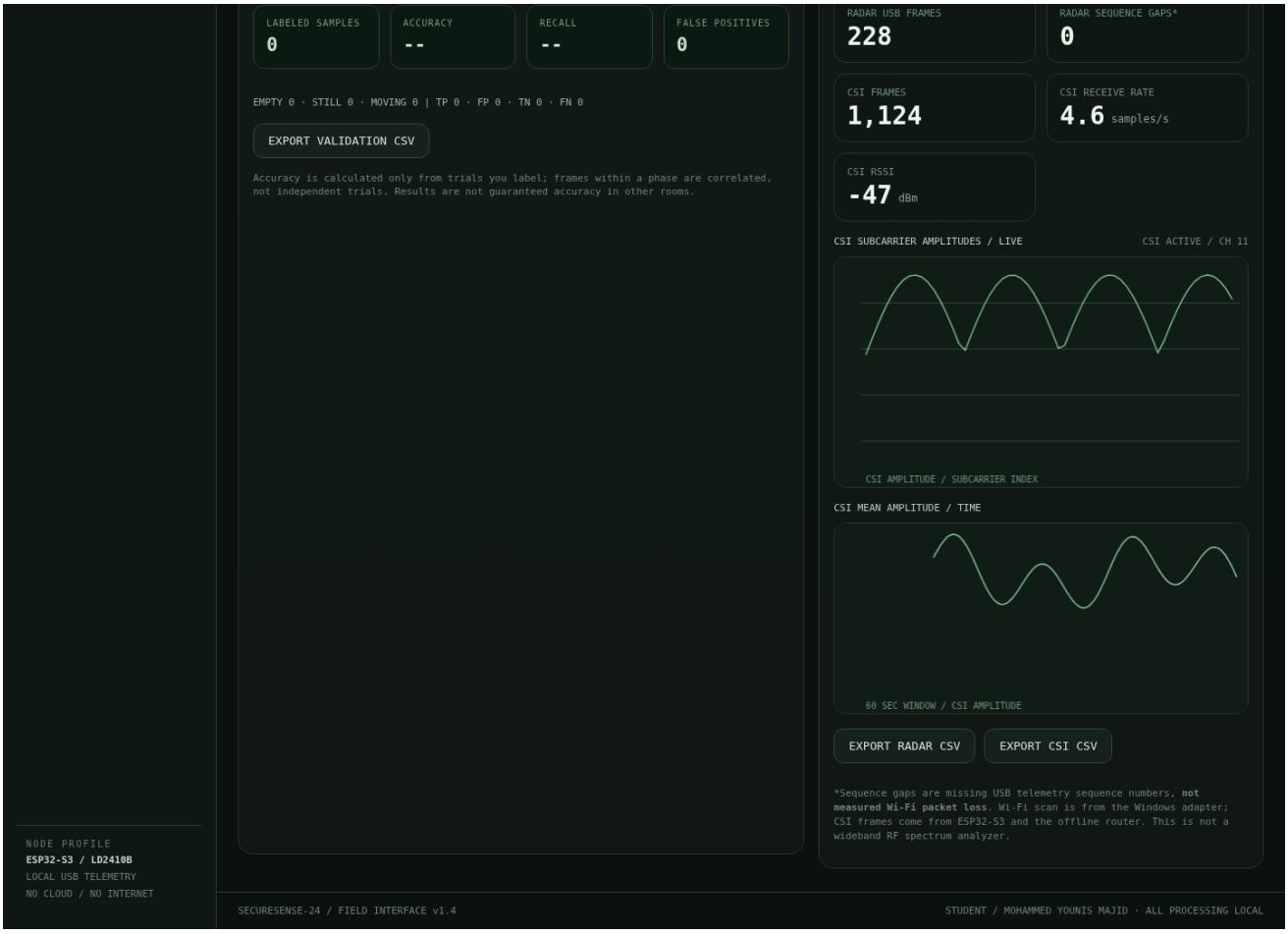



*Figure 6. Quiet calibration, motion-score interface, and operator-labeled validation controls (simulated UI preview).*

The evaluation panel intentionally requires operator input for EMPTY, STILL and MOVING. Summary metrics are meaningful only after the real three-stage field protocol is completed and CSV results have been preserved.

# 18 | Application Interface: Cryptography

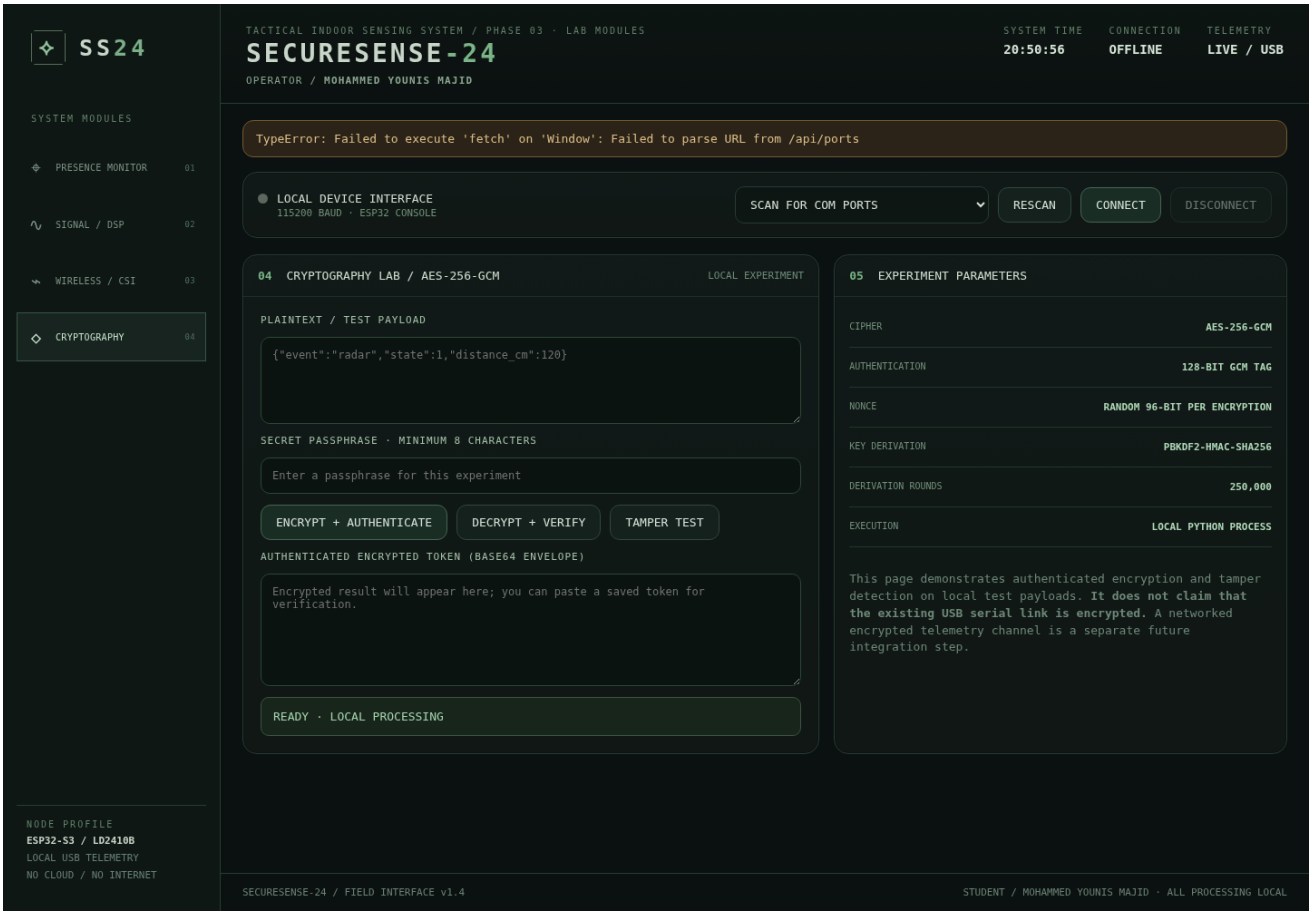



*Figure 7. Cryptography interface for AES-256-GCM, password-based derivation, verification, and optional live telemetry import (offline UI preview).*

The cryptography tab is a controlled authenticated-encryption laboratory. It can use a snapshot of live sensor telemetry as plaintext; it does not encrypt the underlying USB stream in real time.

# 19 | Student-Observed Field Evidence

The following photographs were supplied during the project's actual development and show the earlier working v1.3 application. They provide qualitative evidence that local router CSI reached Windows alongside radar data, and that the DSP and Wireless pages displayed nonempty changing signals. The photographed interface is not evidence of v1.4 classifier accuracy.




*Figure 8. Student-supplied photograph of the running Wireless/CSI dashboard (v1.3; field evidence).*

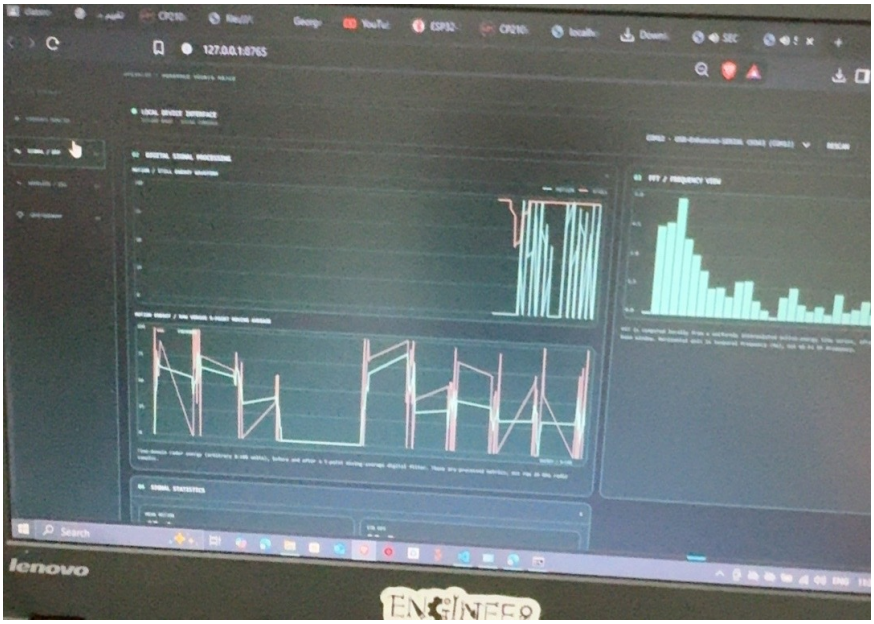



*Figure 9. Student-supplied photograph of time-domain and FFT views during real testing (v1.3; field evidence).*

# 20 | MATLAB Appendix: Measured CSV Analysis

## 20.1 Reproduce motion-classification metrics

The Windows app exports `securesense24_evaluation.csv` with a phase label and CSI prediction per valid sample. The following compact script reproduces confusion-matrix statistics without inventing data:

```
V = readtable('securesense24_evaluation.csv');
truth = strcmp(string(V.operator_label), 'MOVING');
pred  = logical(V.csi_motion_detected);
TP=sum( truth &  pred); TN=sum(~truth & ~pred);
FP=sum(~truth &  pred); FN=sum( truth & ~pred);
safe = @(a,b) 100*a/max(b,1);
Accuracy=safe(TP+TN, TP+TN+FP+FN);
Precision=safe(TP,TP+FP); Recall=safe(TP,TP+FN);
Specificity=safe(TN,TN+FP); F1=safe(2*TP,2*TP+FP+FN);
disp(table(TP,FP,TN,FN,Accuracy,Precision,Recall, ...
    Specificity,F1));
```

**Important correction:** in this compact code, `max(denominator,1)` is used only to avoid MATLAB division errors. For a formal paper, denominators equal to zero must be reported as **NaN/not applicable** rather than as a valid zero-percent statistic.

## 20.2 Visualize labeled motion against the reported score

```
figure; plot(V.time_monotonic_sec, V.csi_score, '.-');
hold on; yyaxis right
stairs(V.time_monotonic_sec, double(truth), 'LineWidth',1.2);
ylabel('Operator-labeled motion (0/1)');
yyaxis left; ylabel('Relative CSI motion index (0-100)');
xlabel('Monotonic experiment time (s)'); grid on
```

MATLAB examples assume a standard MATLAB version with `readtable`, `movmean` and `fft`. The windowed FFT illustration uses `hann`, which may require the Signal Processing Toolbox; it can be replaced by the explicit cosine formula shown in Chapter 4.

# 21 | Limitations, Conclusion & Recommendations

## 21.1 Technical limitations

- The radar module does not expose target angle and cannot reliably count multiple people in this implementation.
- CSI magnitude variations are not specific to humans; they may reflect environmental changes, multipath, or receiver conditions.
- Published firmware CSI vectors are selected magnitude slots, not complete calibrated raw I/Q channel estimates.
- A temporal FFT is not a hardware RF spectrum analyzer. Short windows and irregular sample timing bound frequency interpretation.
- The AES-GCM laboratory demonstrates secure payload processing but does not protect the raw USB stream or authenticate the local web service.
- No validated confusion matrix or generalizable sensitivity/specificity estimate is available until real v1.4 sessions are recorded.

## 21.2 Conclusion

The student-operated prototype has demonstrated concurrent 24 GHz radar telemetry and 2.4 GHz Wi-Fi CSI delivered to an offline Windows dashboard. The final v1.4 software integrates digital filtering, short-window frequency analysis, robust CSI variation detection, conservative two-sensor fusion, operator-labeled evaluation, and a cryptographic payload experiment. This is a coherent cross-course platform, but it remains a laboratory prototype. Its practical value is the reproducible architecture and transparent separation between measurements, illustrative mathematics, automated tests, and claims requiring field validation.

## 21.3 Recommended completion checklist

- Perform the 12-second quiet calibration with a stable router, fixed device position and no moving participants.
- Run all three 20-second phases in multiple independent sessions, then export all three CSV datasets.
- Report both session-level and frame-level results, and retain the number of repetitions and room geometry.
- Capture final v1.4 *live* screenshots before academic submission to replace illustrative UI previews where appropriate.

# References & Source Provenance

[1] Espressif Systems, *ESP32-S3 ESP-IDF Programming Guide - Wi-Fi Vendor Features / CSI*. https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-guides/wifi-driver/wifi-vendor-features.html

[2] M. Dworkin, *NIST SP 800-38D: Recommendation for Block Cipher Modes of Operation: GCM and GMAC*, National Institute of Standards and Technology, 2007. https://doi.org/10.6028/NIST.SP.800-38D

[3] M. S. Turan et al., *NIST SP 800-132: Recommendation for Password-Based Key Derivation, Part 1*, National Institute of Standards and Technology, 2010. https://doi.org/10.6028/NIST.SP.800-132

[4] Northern Technical University, official university identity and public webpages. https://ntu.edu.iq/

[5] Technical Engineering College for Computer and AI / Mosul, official college webpage. https://caitc.ntu.edu.iq/

[6] Project local source archive, *SECURESENSE-24 v1.4 FINAL*: ESP-IDF firmware, Python backend, CSI classifier, browser UI, automated tests and technical notes (user-provided project session).

[7] Student-supplied Windows and ESP-IDF photographs of the working v1.3 radar / CSI prototype, October 2026, used as qualitative field evidence in Figures 8-9.

## DOCUMENT INTEGRITY & TEST STATUS

The official NTU and college emblems were reused from an earlier university laboratory report containing the original logos and matched to the institution's online visual identity; they were not newly generated. The report does not expose any router password, biometric data, or personally identifying wireless MAC addresses. Figures 1-7 are identified simulated/offline application previews, while Figures 8-9 are photographs from the student's working setup. No unprovided accuracy result is reported.

**Prepared for:** Digital Signal Processing (DSP), Wireless Networks, and Cryptography.
**Student:** Mohammed Younis Majid.
**Document version:** October 2026 / SECURESENSE-24 v1.4.